Smart Flow Commute: An Intelligent Urban Commuting Assistant Using Graph-Based Route Planning

Sarvesh R, Rittick V

Department of Computer Science and Engineering, Rajalakshmi Engineering College, Thandalam, Chennai, India | Faculty Guide: Mrs. Deepa | [Email Address]

Abstract

Commuters in large cities must choose routes while travel conditions change because of congestion, incidents, road works, weather and incomplete information. Selecting a route from habit or from a static map can make it difficult to compare alternatives, understand why one journey is recommended, and recognise when the available information is unreliable. This paper presents Smart Flow Commute, a web-based commuting assistant that combines trip input, graph-based route search, ranked journey options, and schedule and alert views in a single interface. The documented reference implementation represents the network as a weighted directed graph, finds minimum-cost journeys with Dijkstra's algorithm or A* using a pluggable cost function (travel time or distance), validates each returned path independently, and reports invalid input and unreachable destinations as distinct statuses. This paper describes the problem context, related research, requirements, the route-cost model, system modules, implementation characteristics, and a feature-level evaluation approach. On a six-node demonstration network, the reference implementation returned the minimum-time journey of 28 min over 9.2 km, and A* returned the same cost as Dijkstra. These are correctness checks on demonstration data, not measurements of live-traffic behaviour. No controlled user study, verified live-data integration or quantitative outcome dataset is available for this paper. Therefore, the paper does not claim that Smart Flow Commute reduces travel time, predicts congestion, or improves commuter outcomes. The system is presented as a practical commuting-support prototype whose usability and real-world effectiveness should be evaluated in future work.

Index Terms— Smart commuting, urban mobility, route planning, graph search, Dijkstra's algorithm, A* search, decision support, web application.

I. INTRODUCTION

A. Background

Urban commuting involves moving through a network of roads, paths, public transport links and interchange points. Travel time varies with peak-hour demand, traffic incidents, road works, weather, public events and differences in road capacity. Commuters often rely on habitual knowledge, static maps or information that may not reflect current conditions, and a route that is normally convenient may become inefficient when conditions change.

Digital tools can help by representing the network formally and comparing candidate journeys. Road networks are commonly modelled as graphs in which junctions are vertices and road segments are edges carrying a cost such as distance or expected travel time [1], [4]. Shortest-path algorithms such as Dijkstra's algorithm [2] and A* search [3] then provide a precise basis for comparing alternatives. This literature motivates a route-planning application built on explicit, testable rules. It does not, however, establish that any particular application will reduce travel time in practice, because that depends on the quality, freshness and coverage of the data supplied to the algorithm.

B. Problem Statement

Commuters face several connected challenges. First, they must identify feasible journeys across a network that may mix walking, bus and metro legs. Second, they need to compare options whose time, distance and number of transfers can conflict. Third, journeys can become invalid or slower when a service is delayed, cancelled or disrupted. Finally, a route recommendation can be misleading if the system hides stale data, unresolved locations or incomplete road information.

The problem addressed in this paper is the design and implementation of a commuting assistant that accepts an origin and destination, computes a minimum-cost feasible journey, presents ranked options with step-by-step directions, distinguishes failure causes, and communicates the limits of its data. Smart Flow Commute is intended to assist journey planning; it is not intended to replace official transport information, local traffic rules, or the traveller's own judgment.

C. Proposed System

Smart Flow Commute is a web application that combines route planning with schedule and alert information. Its main areas, as shown in screenshots of the project interface, include a dashboard, a route planner, a schedule and alerts view, and ranked journey options with a step-by-step itinerary. The route planner accepts an origin and destination and returns one or more journeys, each summarised by total time and distance.

The central route mechanism models the network as a graph G = (V, E), assigns each edge a travel mode, time and distance, and searches for the path of minimum total cost. The reference implementation supports Dijkstra's algorithm and A* within one search function, a replaceable cost function, an alternatives routine that re-runs the search under different cost functions, and an independent validator that checks every consecutive pair of nodes on the returned path.

The system distinguishes three outcomes of a request: a valid route, an invalid input (for example an empty or unknown location), and a destination that cannot be reached. These statuses are application-defined and are not predictions of real-world travel time.

D. Objectives

1. To provide a single interface for entering a trip, viewing ranked journey options, and consulting schedule and alert information.
2. To model the network as a weighted directed graph and compute minimum-cost journeys with a transparent cost function.
3. To compare candidate routing methods and select a baseline suitable for a first implementation.
4. To validate returned paths independently and report invalid input and unreachable destinations as distinct statuses.
5. To present time, distance and itinerary details so that users can see why an option was recommended.
6. To document the implemented workflows and identify appropriate functional and empirical evaluation methods.

E. Contributions and Scope

The first contribution is the integration of trip input, graph-based route search, ranked alternatives, and schedule and alert views within one commuter-oriented application. The second is a transparent route-cost model that supports travel time, distance, and a documented weighted combination of criteria. The third is an independent path validator

with explicit failure statuses and known-answer tests. These contributions concern system design and implementation; they should not be interpreted as a new routing algorithm or a validated traffic-prediction model.

The scope of this paper is feature-level system design and implementation. The reference implementation is a TypeScript routing module with a React and Tailwind front-end component, and it operates on a small demonstration network held in memory. The deployed web application is hosted on the Lovable platform; its exact framework, data sources, map or geocoding provider, and any live-traffic integration have not been independently verified and are not asserted here. This distinction prevents demonstration behaviour from being incorrectly described as a live traffic service.

F. Organization of the Paper

Section II reviews relevant background. Section III presents requirements and the route-cost methodology. Section IV describes the system architecture and module interactions. Section V documents implementation characteristics. Section VI outlines feature-level testing and discusses the limits of the available evidence. Section VII concludes the paper and identifies future work.

II. RELATED WORK

A. Graph-Based Route Planning

Road networks are conventionally represented as graphs. Intersections or stops form vertices, road or service segments form edges, and each edge may carry a cost such as distance, expected travel time or toll [1], [4]. A path is a sequence of connected edges from origin to destination, and this representation allows the cost of alternative paths to be compared precisely. For Smart Flow Commute, the design implication is that route quality can be reasoned about and tested on small graphs with known answers, while claims about real-world benefit must remain tied to measured outcomes.

B. Shortest-Path Algorithms

Dijkstra's algorithm finds a minimum-cost path in a graph with non-negative edge weights [2]. A* search ranks nodes by $f(n) = g(n) + h(n)$, where $g(n)$ is the known cost from the origin and $h(n)$ estimates the remaining cost, and it can reduce exploration when the heuristic does not overestimate [3]. Breadth-first search suits unweighted graphs but

ignores differing travel times [1]. Smart Flow Commute applies these standard methods rather than proposing a new one, and its reference implementation is comparatively interpretable: a user can inspect the cost of each leg of a recommended journey.

C. Dynamic Routing and Traffic Information

Static routing assumes edge costs remain fixed during a search, whereas real conditions change after a route has been calculated. Dynamic routing therefore requires timestamped observations, an update policy, and a rule for deciding when a new route should be computed, so that small or noisy fluctuations do not cause repeated route changes. Dynamic data should carry a source, a timestamp and a quality indicator. When no live feed is connected, the output should be described as based on static or demonstration data, as the reference implementation does.

D. Research Gap and Positioning

The cited literature establishes the suitability of graph models and shortest-path algorithms for route planning, but it does not establish the effectiveness of Smart Flow Commute itself. The system′s practical focus is the combination of route search, ranked alternatives, explicit failure statuses and schedule and alert views in a single commuter workflow. The paper′s contribution is therefore positioned as an implemented system design with explicit routing logic and independent validation. A comparison against existing navigation tools or a controlled commuter study would be required to establish whether this integration offers measurable advantages.

III. REQUIREMENTS AND METHODOLOGY

A. Intended Users and Requirements

The intended users are commuters who need to plan a journey across a network. The primary functional requirements are: (1) enter or select an origin and destination; (2) validate the request and report missing or unknown locations; (3) compute a feasible minimum-cost journey; (4) present ranked options with total time, distance and step-by-step legs; (5) report unreachable destinations explicitly; and (6) view schedule and alert information.

Non-functional considerations include usability, responsiveness, determinism for fixed inputs, explainability of recommendations, accessibility and appropriate handling of location data. Because the reference implementation uses an in-memory demonstration

graph, it does not provide live data, account-based storage or multi-device synchronisation.

B. Route-Cost Model

Let the network be a directed graph G = (V, E), where V is the set of junctions and stops and E is the set of traversable segments. Each edge e has a non-negative cost w(e), a travel mode (walk, bus or metro), a time t(e) in minutes and a length d(e) in kilometres. Given an origin o and destination d, the goal is to find a feasible path P that minimises the total cost:

```
C(P) = \sum_{e \in P} w(e)
```

For a travel-time objective w(e) = t(e), and for a distance objective w(e) = d(e). A multi-criteria objective can be written as:

```
C(P) = \alpha T(P) + \beta D(P) + \gamma R(P)
```

where T is travel time, D is distance, R is a documented route penalty (for example for transfers), and the weights are documented and normalised. This weighted form is a design option; the reference implementation currently provides the time and distance cost functions and a pluggable interface for others.

Hard constraints determine feasibility: edges must be connected, one-way restrictions respected, and confirmed closures excluded. Preferences influence ranking only and never override a safety-critical or legal restriction. The weights and any thresholds are design choices that have not been calibrated through a controlled commuter study.

C. Algorithm Selection

Table I compares the candidate methods. Dijkstra's algorithm is adopted as the baseline because edge costs are non-negative and the graph is small. With a binary heap its running time is commonly described as O((|V| + |E|) log |V|). A* is supported by supplying a heuristic: the reference implementation scales the straight-line distance between node coordinates by a minimum time per unit distance so that the estimate does not exceed the true remaining time. When no heuristic is supplied, the same function behaves as Dijkstra's algorithm.

TABLE I. Candidate route-planning methods

Method	Best use	Strength	Limitation
Breadth-first search	Unweighted graphs	Simple and complete	Ignores varying edge costs
Dijkstra	Non-negative weighted graphs	Finds minimum-cost path	May explore many nodes
A*	Weighted graph with a valid heuristic	Reduced exploration	Depends on heuristic validity
Dynamic re-planning	Changing edge costs	Reacts to updated information	Requires fresh, reliable data

D. Alternatives and Ranking

The alternatives routine re-runs the search under different cost functions (time, then distance) and removes duplicate paths, so the interface can show distinct options only when they differ. Options are ranked by the documented criterion, and the first is labelled as the best route under that criterion. Because time, distance and transfers can conflict, the interface should not hide a long detour behind a single score and should explain what the ranking means.

E. Validation and Failure Statuses

Every request is validated before search. An empty origin or destination, or a location absent from the network, returns an invalid-input status. If the destination is unreachable, the search returns a no-route status with no path. For a successful search, an independent validator confirms that every consecutive pair of nodes on the path is joined by an existing edge. These cases are reported separately rather than collapsed into a generic error, so the user can revise the request.

IV. SYSTEM DESIGN AND ARCHITECTURE

A. Logical Architecture

At the logical level, Smart Flow Commute can be divided into four layers: (1) the user-interface layer, (2) the application-logic layer, (3) the network-data layer, and (4) the result and feedback layer. The user interface contains the dashboard, route planner, and

schedule and alerts views. Application logic manages request validation, route search, cost evaluation, alternative generation and path validation. The data layer holds the road and transit graph, edge attributes and, where available, schedule and alert records. The result layer converts the search output into ranked journeys, itineraries and status messages.

In the reference implementation, these responsibilities are realised as a TypeScript routing module and a React component operating on an in-memory demonstration graph. The logical layers describe responsibilities, not independently deployed services. They should not be interpreted as evidence that the deployed application uses a separate production database, a remote traffic service or a cloud deployment.

TABLE II. Logical view of Smart Flow Commute (Fig. 1)

Layer	Components
Presentation layer	Dashboard • Route planner • Schedule and alerts • Journey options and itinerary
Application logic	Input validation • Dijkstra / A* search • Cost function • Alternatives • Path validator
Network and data	Nodes and coordinates • Edges (mode, minutes, km) • Schedule and alert records (if integrated)
Outputs	Ranked journeys • Total time and distance • Step-by-step legs • Status and error messages

Fig. 1. Logical view of the main responsibilities. The diagram is conceptual and does not imply separate deployed services.

B. Main Modules

Dashboard. Summarises network status and provides navigation to key workflows, as shown in the project screenshots.

Route planner. Accepts an origin and destination, validates the request and displays ranked journey options with total time and distance.

Itinerary view. Lists the legs of a selected journey, for example a walk, a bus ride, a metro ride and a final walk, each with its duration and distance.

Schedule and alerts. Shows timetable status (on time, delayed or cancelled) and disruption notices, with per-route alert toggles. Its data source has not been independently verified.

Routing engine. Performs the graph search, applies the selected cost function and returns a structured result with an explicit status.

Result validator. Checks path continuity independently of the search so that a shared defect is less likely to go unnoticed.

C. Data Flow

A typical request begins when the user selects an origin and destination. The application validates both, resolves them to graph nodes, and runs the search. The structured result is passed to the validator and then to the interface, which renders the ranked options and itinerary or an explanatory message. If dynamic data were integrated, updated edge costs would trigger re-evaluation, but this is future work and is not claimed for the current build.

D. Design Considerations

Explicit rules and a transparent cost model make results explainable, because each leg of a journey carries its own time and distance. The trade-off is that a simplified graph cannot fully model turn restrictions, closures, access rules, crowding, transfer waiting time or differences between travel modes. A route can be mathematically correct and still be a poor journey if the graph or costs are stale or incomplete. The interface should therefore state whether a value is measured, estimated or demonstration data, and it should avoid presenting estimated arrival times as guarantees.

V. IMPLEMENTATION

A. Prototype Technologies and Data Model

The reference implementation is a TypeScript module with a React and Tailwind front-end component, consistent with the web stack commonly produced by the Lovable application builder. The graph is stored as a mapping from each node to a list of outgoing edges. Each edge records its destination node, travel time in minutes, distance in kilometres, travel mode and a display label. A separate coordinate table supports the A* heuristic. The deployed application's own framework and dependencies must be confirmed from its exported source before being reported as implemented behaviour.

B. Route Search and Ranking

The search function maintains a table of best-known costs, a record of each node's predecessor edge, a set of settled nodes and a binary min-heap priority queue. It repeatedly removes the node with the lowest priority, relaxes its outgoing edges and stops when the destination is settled. The priority is the accumulated cost, plus the heuristic estimate when A* is used. The path and itinerary are then reconstructed by following predecessor edges back from the destination, and totals are computed from the edges on the path.

C. Validation and Error Handling

The function returns a structured result containing a status, the node path, the edge steps, total minutes and total kilometres. Input validation precedes any search, and an unreachable destination returns a no-route status rather than an invalid path. The user interface displays an error message with an alert role for invalid input, labels its data as demonstration data, and renders each option with its time, distance and legs. The path validator is applied to each displayed option.

D. Scope of Intelligence

Although the project is positioned as an intelligent commuting assistant, the documented routing logic is deterministic graph search. It does not demonstrate that a machine-learning model has been trained, that traffic is predicted, or that live sensor, camera or GPS data are connected. Any such capability, and any separate backend or live-data integration, should be described as implemented only after its active behaviour and dependencies are verified in the final build.

E. Privacy and Deployment Boundaries

Origin and destination data can reveal home, workplace and routine, so a deployment should request only the location information needed for the trip, explain any location permission, and avoid retaining trip history without a clear purpose and notice. A production system would need secure transport, protection of API credentials outside public client code, input validation, graceful handling of service failures, and logging that avoids retaining precise location histories. If a third-party map, geocoding or traffic provider is introduced, its terms, rate limits, update frequency and failure behaviour should be documented.

VI. TESTING AND EVALUATION

A. Evaluation Approach

The evidence available for this manuscript consists of a reference routing implementation with automated known-answer checks and screenshots of the project interface. It does not include a formal test report for the deployed site, a measured user study, or a benchmark dataset. Accordingly, this section reports only the checks that were executed against the reference implementation and describes the appropriate scope for the remaining evaluation. No pass rate, timing result, user-satisfaction score or travel-time saving is reported.

Functional evaluation should verify that each workflow produces the expected output. Algorithmic evaluation should compare returned costs with known optima on small graphs and confirm that A* matches Dijkstra. Performance evaluation should use larger synthetic graphs, recording graph size, algorithm configuration and environment, and separating search time from geocoding, network and rendering time. Usability evaluation should examine whether commuters can enter a trip, interpret the result and recover from invalid input.

B. Feature-Level Test Scenarios

TABLE III. Verification matrix for the reference implementation

ID	Test scenario	Expected behavior	Evidence/status
T1	Valid origin and destination (Home to Office)	A valid route is returned	Passed in reference self-test
T2	Missing origin	Invalid-input status	Passed in reference self-test
T3	Missing destination or unknown location	Invalid-input status	Same validation branch; verify in final build
T4	Disconnected destination (Office to Home)	No-route status, no path	Passed in reference self-test
T5	Known weighted graph	Minimum-cost path (28 min)	Passed in reference self-test

ID	Test scenario	Expected behavior	Evidence/status
T6	Path continuity	Every consecutive edge exists	Passed in reference self-test
T7	A* versus Dijkstra	Equal total cost	Passed in reference self-test
T8	Repeated identical request	Identical output for fixed data	Deterministic by design; verify in final build
T9	Narrow or mobile viewport	Controls remain usable	Pending; deployed application
T10	External service unavailable	Clear failure or documented fallback	Pending; applicable only if a service is integrated

Table III. Entries marked pending should be replaced with observed outcomes from the final build's test log.

C. Worked Example

The reference implementation was run on a six-node demonstration network (Home, Central Stop, Junction Square, Ring Road, Tech Park and Office). Two feasible journeys exist from Home to Office (Table IV). The planner returned Path A, the minimum-time route, consisting of a 4 min walk, a 14 min bus ride, an 8 min metro ride and a 2 min walk.

TABLE IV. Candidate journeys on the demonstration network (Home to Office)

Path	Legs	Time (min)	Distance (km)
A (selected)	Walk, Bus 12, Metro Blue, Walk	28	9.2
B	Walk, Bus 04 via Ring Road, Bus 04, Walk	41	10.5

D. Interpretation of Evaluation Evidence

The implementation supports a coherent workflow from trip input to ranked journeys, with explicit statuses and independent path validation. The deterministic nature of the search makes it suitable for unit testing with controlled inputs. Nevertheless, correct execution on a six-node graph demonstrates correctness of the logic only. It does not

establish scalability, response time, live-traffic behaviour or the accuracy of estimated travel times, and a route being displayed does not prove it is optimal in the real world. Those questions require larger benchmarks, real data and commuter-centred evaluation.

A subsequent evaluation could recruit volunteers to plan a set of sample journeys and assess task completion, error frequency, perceived usability and the clarity of results. A separate study would be needed to compare estimated and observed travel times over a defined period and geography. Any such study should define participants, instruments, consent procedures and analysis methods before collecting data.

E. Limitations

- The reference implementation uses a small in-memory demonstration graph; live traffic, GPS tracking and predictive models are not demonstrated.
- Cost weights and ranking criteria are design choices and have not been calibrated through a controlled study.
- The graph omits turn restrictions, closures, access rules, transfer waiting time and crowding.
- The deployed application's framework, data sources and integrations have not been independently verified.
- Estimated times are uncertain and should not be treated as guarantees; users should plan before departure and not use the interface while driving.
- No controlled user study or quantitative outcome dataset is available for this manuscript.

VII. CONCLUSION AND FUTURE WORK

This paper presented Smart Flow Commute, a commuting assistant that combines graph-based route planning with ranked journey options and schedule and alert views. The documented implementation includes a weighted graph model, Dijkstra and A* search with a pluggable cost function, alternative generation, explicit failure statuses and an independent path validator. On a demonstration network it returned the expected minimum-time journey, and A* matched Dijkstra's cost.

The main contribution is an integrated, interpretable route-planning workflow rather than a claim of proven travel-time savings or a newly trained prediction model. At the same time, the demonstration data, simplified graph and absence of verified live feeds impose

important limitations. No claim is made that the system reduces commute time or predicts congestion.

Future work should begin with verifying the deployed build against its source and documenting each implemented feature, followed by executing the full test plan and collecting repeatable timing data across graph sizes. A dependable, timestamped traffic or incident source could then be integrated with a defined fallback, and explicit, reversible preferences and accessibility improvements could be added. A production version should provide secure data handling and clear disclosure of how location data is processed. Finally, a controlled study comparing estimated with observed travel times would be necessary to determine whether Smart Flow Commute improves journey planning compared with existing methods.

REFERENCES

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[3] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[4] R. E. Tarjan, *Data Structures and Network Algorithms*. Philadelphia, PA, USA: SIAM, 1983.

[5] OpenStreetMap contributors, "OpenStreetMap." [Online]. Available: https://www.openstreetmap.org/

[6] Smart Flow Commute project website. [Online]. Available: https://smart-flow-commute.lovable.app/. Accessed: Oct. 23, 2026.